

Lập trình .NET (ASP.NET Core Web API)

**Buổi 1: Giới thiệu Học Phần
& Ôn Tập C#**



1. Lý thuyết

- 1.1. Giới thiệu học phần
- 1.2. Tổng quan .NET, CLR, BCL, .NET SDK
- 1.3. Ôn tập OOP, Collections, Generic, LINQ, Lambda, Exception, async/await

2. Thực hành

- 2.1. Cài đặt môi trường
- 2.2. Tạo Console Application
- 2.3. Quản lý danh sách sinh viên với LINQ

1.1. Giới thiệu học phần

- **Mục tiêu**
- **Công nghệ sử dụng**

1.1. Giới thiệu học phần (tt)

Mục tiêu

Xây dựng được một ứng dụng Web API hoàn chỉnh bằng ASP.NET Core.

Công nghệ sử dụng

Framework: .NET 8 (LTS)

IDE: Visual Studio 2022

Database: SQL Server 2022

API Testing: Postman / Swagger

ORM: Entity Framework Core

1.2. Tổng quan .NET, CLR, BCL, .NET SDK

.NET là gì?

Các thành phần chính.

1.2. Tổng quan .NET, CLR, BCL, .NET SDK

.NET là gì?

Nền tảng phát triển phần mềm đa nền tảng — Windows, Linux, macOS

Hỗ trợ nhiều loại ứng dụng: Web API, Console, Desktop, Mobile, Cloud

Mã nguồn mở, được phát triển bởi Microsoft và cộng đồng

Các thành phần chính.

.NET
SDK

Runtime

CLR

Common Language Runtime

BCL

Base Class Library

1.2. Tổng quan .NET, CLR, BCL, .NET SDK (tt)

Kiến trúc .Net

CLR, BCL và .NET SDK

Ba thành phần cốt lõi — mỗi thành phần một vai trò riêng

CLR

Common Language Runtime

- Môi trường thực thi mã .NET
- Quản lý bộ nhớ qua Garbage Collector
- Bảo mật (CAS, code access security)
- Xử lý ngoại lệ runtime thống nhất

EX GC.Collect() → *giải phóng object không dùng*

BCL

Base Class Library

- Tập hợp thư viện dựng sẵn sẵn sàng dùng
- System.IO — xử lý file và stream
- System.Collections — List<T>, Dictionary<K,V>
- System.Linq — truy vấn dữ liệu

EX using System.Linq;

.NET SDK

Software Development Kit

- Bộ công cụ xây dựng & chạy ứng dụng .NET
- Roslyn compiler (C# → IL)
- dotnet CLI — tạo, build, run, test
- Build tools và templates tích hợp

EX dotnet new webapi -n MyApi



Ghi nhớ: SDK = tools để BUILD | Runtime = môi trường để CHẠY

OOP trong C#

Bốn trụ cột OOP — và vai trò trong Web API

Khái niệm	Ứng dụng trong Web API
Class / Object	Model, Controller, Service
Interface	Contract cho Repository, Service
Inheritance	Base Controller, Base Entity
Encapsulation	Property với getter/setter

OOP trong C#

Ví dụ nhanh:

```
public interface IStudentService
```

```
{  
    List<Student> GetAll();  
}
```

```
public class StudentService : IStudentService
```

```
{  
    public List<Student> GetAll() => _students;  
}
```

 **Best Practice:** Lập trình theo Interface – không phụ thuộc vào class cụ thể.

Collections & Generic

Kiểu dữ liệu phổ biến và sức mạnh của Generic

Collection	Dùng khi
List<T>	Danh sách có thứ tự, cần thêm/xóa
Dictionary<K, V>	Tra cứu nhanh theo key
IEnumerable<T>	Trả dữ liệu từ Repository/Service
IQueryable<T>	Build query động, chưa thực thi
HashSet<T>	Tập hợp phần tử duy nhất

1.3. Ôn tập C# (tt)

Generic – Ví dụ thực tế:

// Response wrapper dùng được cho mọi kiểu dữ liệu

```
public class ApiResponse<T>
{
    public bool Success { get; set; }
    public T Data { get; set; }
    public string Message { get; set; }
}
```

 **Ghi nhớ:** “IEnumerable<T>” dùng để trả dữ liệu. “List<T>” dùng khi cần thao tác.

LINQ & Lambda



1.3. Ôn tập C# (tt)

LINQ là gì?

Language Integrated Query – truy vấn dữ liệu ngay trong C#

Dùng để lọc, sắp xếp, biến đổi danh sách

1.3. Ôn tập C# (tt)

Ví dụ thực tế:

```
var students = new List<Student>
{
    new Student { Id = 1, Name = "An", GPA = 3.5 },
    new Student { Id = 2, Name = "Bình", GPA = 2.8 },
    new Student { Id = 3, Name = "Chi", GPA = 3.9 },
};

// Lọc, sắp xếp, chiếu dữ liệu
var result = students
    .Where(s => s.GPA >= 3.0)
    .OrderByDescending(s => s.GPA)
    .Select(s => new { s.Name, s.GPA })
    .ToList();
```

Exception Handling

Tại sao cần xử lý ngoại lệ?

- Ứng dụng không crash khi gặp lỗi bất ngờ
- Trả thông báo lỗi rõ ràng cho người dùng API

1.3. Ôn tập C# (tt)

Cấu trúc:

```
try {  
    var student = GetStudentById(id);  
} catch (KeyNotFoundException ex) {  
    Console.WriteLine($"Không tìm thấy: {ex.Message}");  
} catch (Exception ex) {  
    Console.WriteLine($"Lỗi hệ thống: {ex.Message}");  
} finally {  
    Console.WriteLine("Hoàn thành.");  
}
```



Lỗi thường gặp: Bắt “Exception” quá chung → mất thông tin lỗi cụ thể.

1.3. Ôn tập C# (tt)

Cấu trúc:

```
try {  
    var student = GetStudentById(id);  
} catch (KeyNotFoundException ex) {  
    Console.WriteLine($"Không tìm thấy: {ex.Message}");  
} catch (Exception ex) {  
    Console.WriteLine($"Lỗi hệ thống: {ex.Message}");  
} finally {  
    Console.WriteLine("Hoàn thành.");  
}
```



Lỗi thường gặp: Bắt “Exception” quá chung → mất thông tin lỗi cụ thể.

1.3. Ôn tập C# (tt)

async / await

Tại sao cần trong Web API?

Web API thường gọi database, file, external API → tốn thời gian

“async/await” giúp không chặn thread → xử lý được nhiều request hơn

1.3. Ôn tập C# (tt)

//  **KHÔNG** nên (blocking)

```
var data = GetDataFromDb();
```

//  **NÊN** (non-blocking)

```
var data = await GetDataFromDbAsync();
```

// Method async phải return Task hoặc Task<T>

```
public async Task<List<Student>> GetAllAsync()  
{  
    return await _dbContext.Students.ToListAsync();  
}
```

 **Lỗi thường gặp:** Dùng `async` nhưng quên `await` → không thực sự bất đồng bộ.

2.1. Cài đặt môi trường

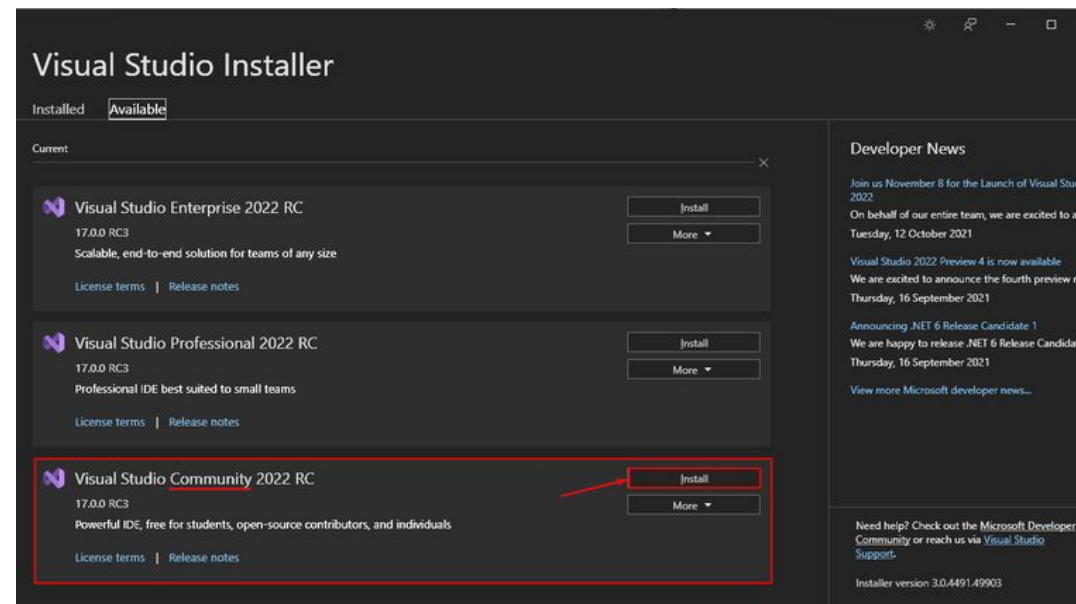
Thiết Lập Môi Trường

1. **.NET 8 SDK** → <https://dotnet.microsoft.com/download>
2. **Visual Studio 2022 Community** (miễn phí)
 - Workload cần chọn: **ASP.NET and web development**
3. **SQL Server 2022 Express** (miễn phí)
4. **Postman** → <https://www.postman.com/downloads>

Kiểm tra cài đặt thành công:

`dotnet --version`

Output mong đợi: 8.x.x



Visual Studio 2022 Installer — chọn workload ASP.NET and web development



Mẹo: Nếu máy yếu, có thể dùng VS Code + C# Dev Kit extension (nhẹ hơn nhiều).

2.2. Tạo Console Application

Demo: Console App Quản Lý Student

Bước 1: Tạo project

```
dotnet new console -n StudentConsoleApp
```

```
cd StudentConsoleApp
```

2.2. Tạo Console Application (tt)

Demo: Console App Quản Lý Student

Bước 2: Định nghĩa Model

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public double GPA { get; set; }
}
```

2.2. Tạo Console Application (tt)

Demo: Console App Quản Lý Student

Bước 3: Quản lý bằng LINQ

```
var students = new List<Student>
```

```
{
```

```
    new Student { Id = 1, Name = "An Nguyễn", GPA = 3.5 },
```

```
    new Student { Id = 2, Name = "Bình Trần", GPA = 2.8 },
```

```
    new Student { Id = 3, Name = "Chi Lê", GPA = 3.9 },
```

```
};
```

```
var topStudents = students
```

```
    .Where(s => s.GPA >= 3.5)
```

```
    .OrderByDescending(s => s.GPA);
```

```
foreach (var s in topStudents)
```

```
    Console.WriteLine($" {s.Name} - GPA: {s.GPA}");
```

2.3. Bài thực hành

Yêu cầu: Xây dựng Console App quản lý danh sách sinh viên.

1. Tạo “List<Student>” với ít nhất 5 sinh viên. In toàn bộ danh sách ra console.

2. Thêm thuộc tính “Major” (ngành học) vào “Student”.

Lọc sinh viên ngành “CNTT” có $GPA \geq 3.0$. Sắp xếp theo GPA giảm dần.

3. Viết method “FindById(int id)”.

Nếu không tìm thấy $\rightarrow$ throw “KeyNotFoundException”.

Bắt exception và in thông báo lỗi thân thiện.

4. Viết class “StudentManager” với các method:

- “AddStudent(Student s)”
- “RemoveStudent(int id)”
- “GetTopStudents(int n)” - trả n sinh viên GPA cao nhất

Tất cả sử dụng LINQ.

5. Chuyển “StudentManager” sang dùng “IStudentManager” interface.

Viết “async Task Main()”, mô phỏng delay 1 giây khi “tải dữ liệu”.

In kết quả dạng bảng căn lề đẹp.



Thank You!